Industry-Oriented Internship Program on
VLSI Design and Semiconductor Engineering

Task 2 - Project Documentation

Combinational and Sequential RTL Design, Finite State Machines, Counters, Shift Registers, Parameterized Verilog Design, Timing Concepts and Advanced RTL Simulation

Name	T R JYOTHIRMAY DIKSHITH
Intern ID	SG2884
College	Dr. Ambedkar Institute of Technology, Bangalore
Program	Sumerix Global - VLSI Design and Semiconductor Engineering
GitHub	JYOTHIRMAYD
Tools	Verilog HDL, Icarus Verilog, GTKWave, Ubuntu Linux, Git
Date	November 2026

Contents

1 Introduction and objectives
2 Tools and project structure
3 Part A - Sequential logic: D flip-flop, register, shift register (Exercise 1)
4 Part B - Counter design (Exercise 2)
5 Part C - Parameterized RTL design (Exercise 3)
6 Part D - Finite state machines: Moore and Mealy traffic light (Exercise 4, mini project)
7 Part E - Timing analysis (Exercise 5)
8 Part F - Testbench development and verification summary (Exercise 6)
9 Exercise 7 - Digital stopwatch controller
10 RTL coding guidelines followed
11 Conclusion and deliverables checklist
A Appendix - RTL source listings

1. Introduction and objectives

Task 1 covered digital electronics fundamentals, the Linux environment, Verilog basics and simple RTL simulation. Task 2 moves to clocked, synchronous design: flip-flops, registers, shift registers, counters, parameterized modules, finite state machines (Moore and Mealy), timing concepts and self-checking testbenches. These blocks are the building blocks of the 32-bit RISC processor planned as the major capstone project.

All designs are written as synthesizable Verilog-2001, simulated with Icarus Verilog and inspected in GTKWave. Every testbench is self-checking and prints **RESULT: PASS** or **RESULT: FAIL**.

Scope of work

Exercise	Design	RTL file(s)	Testbench
1	D flip-flop, 8-bit register, shift register (sync and async reset)	d_flip_flop.v, register_n.v, register8.v, shift_register_n.v	tb_d_flip_flop, tb_register, tb_shift_register
2	4-bit up, down, mod-10, ring and Johnson counters	up_counter4.v, down_counter4.v, mod10_counter.v, ring_counter4.v, johnson_counter4.v	tb_counters
3	Parameterized N-bit register, counter, shift register	register_n.v, counter_n.v, shift_register_n.v	tb_param
4 / Mini project	Moore and Mealy traffic light controllers	traffic_light_moore.v, traffic_light_mealy.v	tb_traffic_moore, tb_traffic_mealy
5	Setup / hold / reset timing analysis	behavioral timing model in testbench	tb_timing_demo
6	Reusable self-checking testbenches	all testbenches	all
7	Configurable digital stopwatch controller	stopwatch_controller.v	tb_stopwatch

2. Tools and project structure

Item	Used
Operating system	Ubuntu Linux (VirtualBox virtual machine)
HDL	Verilog HDL (IEEE 1364-2005 subset, compiled with iverilog -g2005)
Simulator	Icarus Verilog 12.0
Waveform viewer	GTKWave 3.3.126
Version control	Git and GitHub
Editor	Visual Studio Code / terminal

Repository layout

```
vlsi-task2-sequential-rtl/
  rtl/          synthesizable Verilog sources (13 files)
  tb/           self-checking testbenches (9 files)
  sim/          VCD waveforms and logs produced by simulation
  docs/         diagrams, screenshots/, documentation, daily progress log
  run_all.sh    compile and run every testbench
  README.md     setup and simulation instructions
```

Running the simulations

```
./run_all.sh                    # all 9 testbenches
./run_all.sh tb_counters        # one testbench
gtkwave sim/tb_counters.vcd     # open the waveform
```

Result of the full regression on the Ubuntu VM: **9 of 9 testbenches reported RESULT: PASS** (Passed: 9, Failed: 0).

3. Part A - Sequential logic (Exercise 1)

A flip-flop stores one bit and updates only on the active clock edge. The D flip-flop copies *d* to *q* on every rising edge of *clk*. The **d_flip_flop** module is based on the starter code and adds a parameter **ASYNC_RESET** that selects the reset style with a generate block:

- **ASYNC_RESET = 1**: *always @(posedge clk or posedge reset)* - reset acts immediately, independent of the clock.
- **ASYNC_RESET = 0**: *always @(posedge clk)* - reset is only sampled on the clock edge (synchronous).

reset	d	Active clock edge	q (next)
1	x	async: any time / sync: at the edge	0
0	0	rising edge	0
0	1	rising edge	1
0	x	no edge	q (holds)

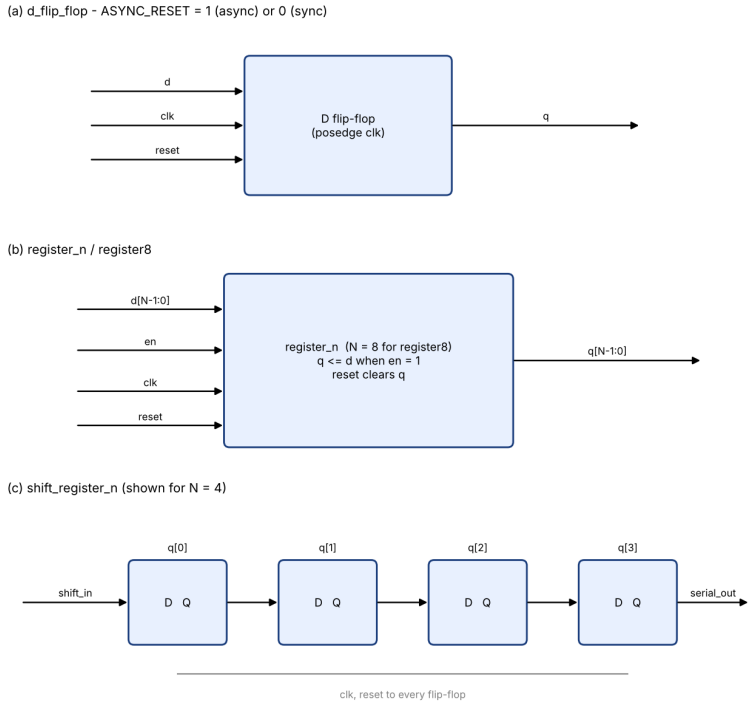


Figure 1: Block diagrams of the D flip-flop, register_n / register8 and shift_register_n.

register_n is an N-bit register with a load enable (*q* <= *d* when *en* = 1). **register8** instantiates it with N = 8. **shift_register_n** shifts left on every clock edge: *q* <= *{q[N-2:0], shift_in}*; the oldest bit leaves through *serial_out* (serial-in/serial-out) and the full word is available on *q* (serial-in/parallel-out).

3.1 Simulation results

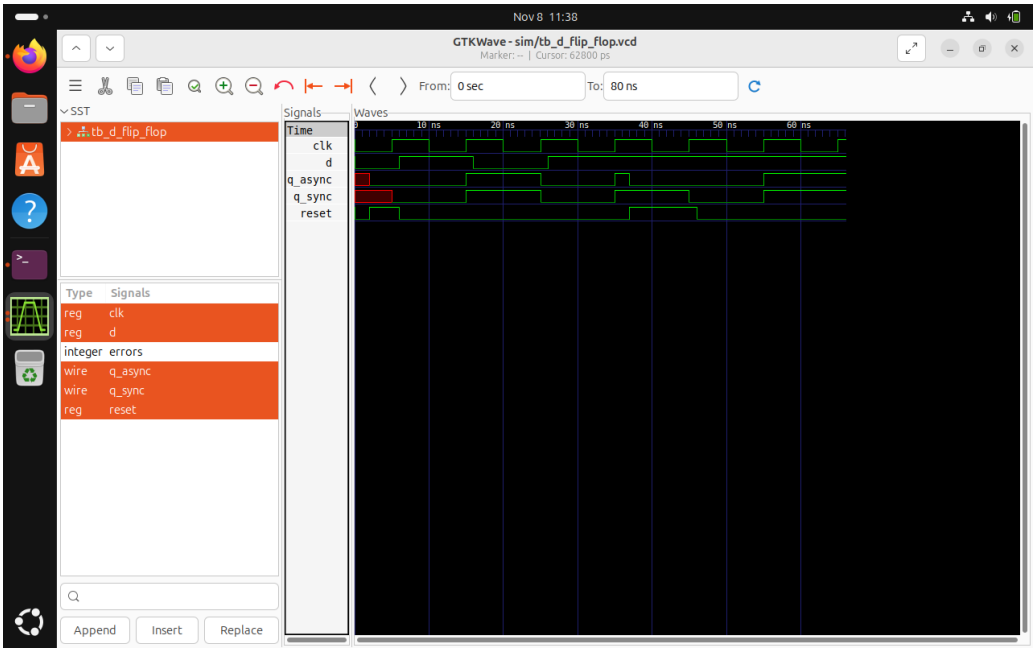


Figure 2: tb_d_flip_flop - async (q_async) versus sync (q_sync) reset.

Both flip-flops start unknown (red, X) until reset is applied. q_async clears as soon as reset rises (about 2 ns), while q_sync clears only on the first clock edge (5 ns). Later, reset is asserted between clock edges at about 37 ns: q_async falls immediately, but q_sync keeps its value until the next rising edge at 45 ns. This is the difference the exercise asks for.

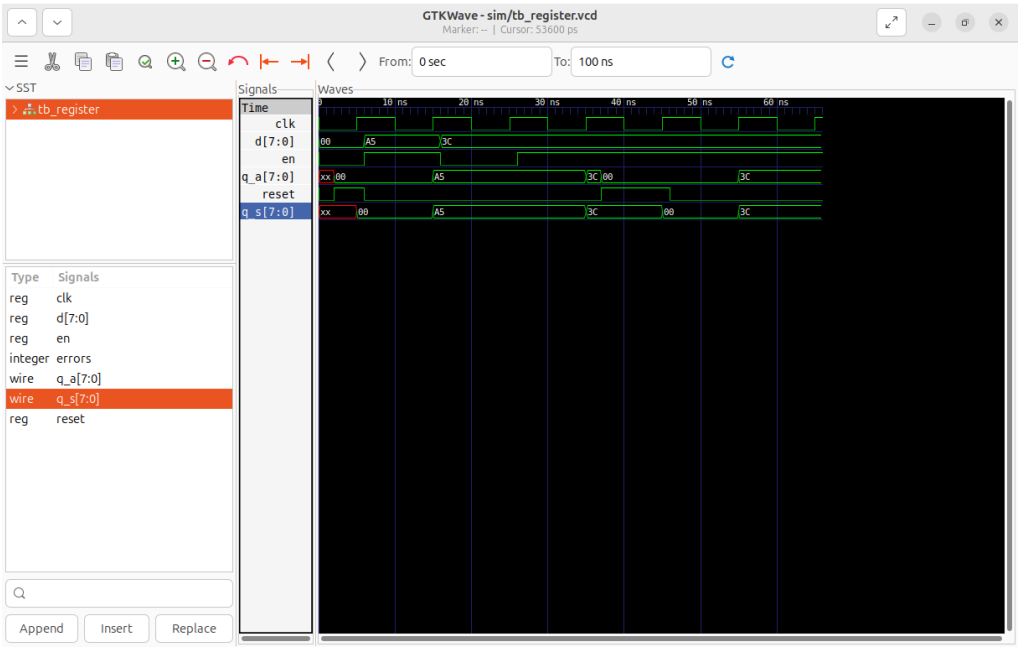


Figure 3: tb_register - 8-bit register with async (q_a) and sync (q_s) reset.

With *en* high the register loads A5 and later 3C on the rising edge. While *en* is low (d = 3C) both outputs hold A5, which confirms the enable. A mid-cycle reset at about 37 ns clears q_a immediately and q_s on the next edge.

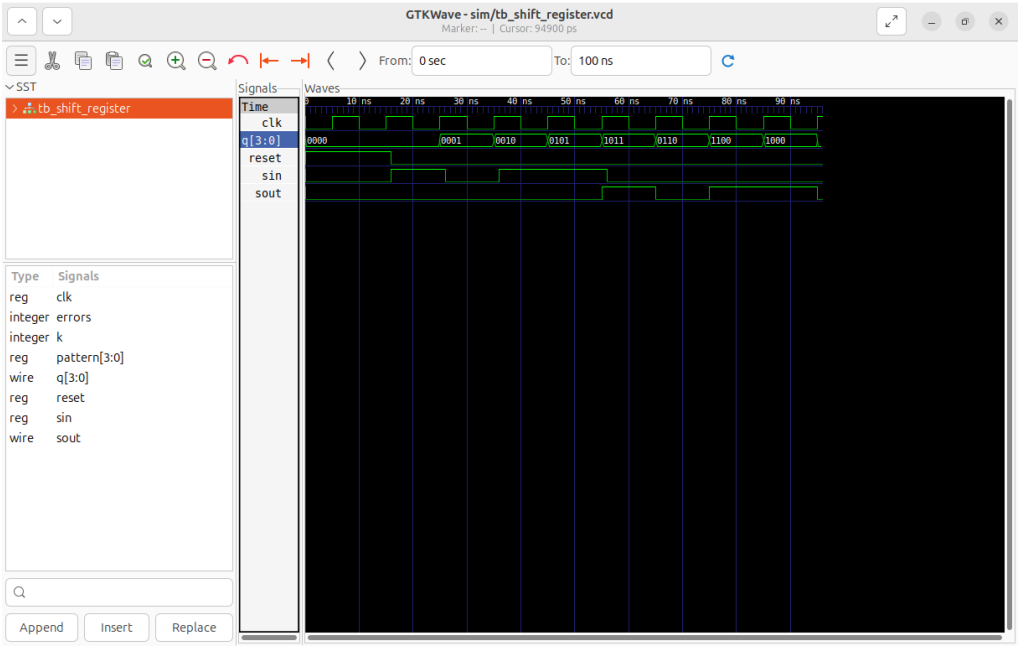


Figure 4: tb_shift_register - 4-bit shift register.

The bits 1, 0, 1, 1 are shifted in on *sin* and *q* builds up as 0001, 0010, 0101, 1011. Zeros are then shifted in and the word moves out through *sout* (0110, 1100, 1000); *sout* goes high whenever a 1 reaches the top bit.

4. Part B - Counter design (Exercise 2)

A counter is a register whose next value is a function of its current value. All five counters use a clock, an asynchronous active-high reset and an enable input, and use non-blocking assignments.

Counter	Reset value	Count sequence	Next-state function
4-bit up	0000	0, 1, 2 ... 15, 0	q + 1 (wraps)
4-bit down	1111	15, 14 ... 0, 15	q - 1 (wraps)
Mod-10	0000	0, 1 ... 9, 0 (terminal count tc at 9)	q == 9 ? 0 : q + 1
Ring (one-hot)	0001	0001, 0010, 0100, 1000	{q[2:0], q[3]}
Johnson (twisted ring)	0000	0000, 0001, 0011, 0111, 1111, 1110, 1100, 1000	{q[2:0], ~q[3]}

A ring counter with N flip-flops has N states; a Johnson counter has 2N states (8 states here), so it divides the clock by 8 and each state differs from the previous by one bit, which avoids decoding glitches.

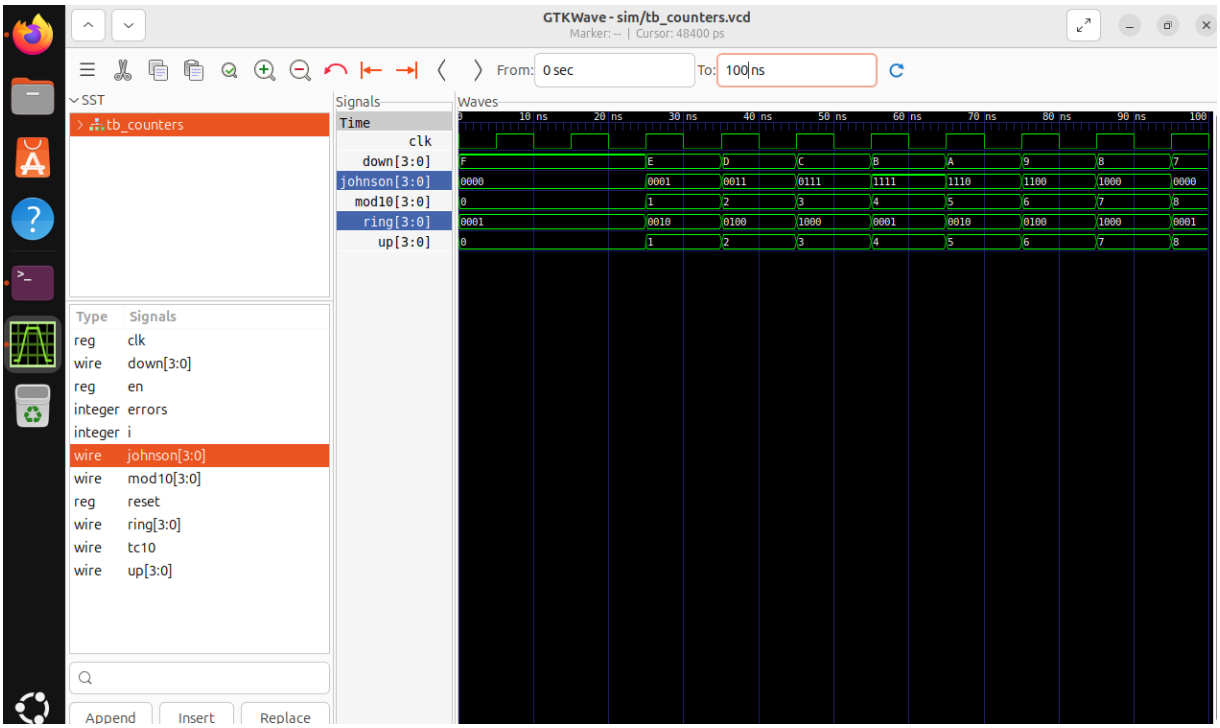


Figure 5: tb_counters - up, down, mod-10, ring and Johnson counters (0 to 100 ns).

After reset is released the counters step on every rising edge: *up* counts 1, 2, 3 ...; *down* starts at F and counts E, D, C ...; *mod10* counts 0 to 9; *ring* shows 0001, 0010, 0100, 1000, 0001; and *johnson* shows 0001, 0011, 0111, 1111, 1110, 1100, 1000, 0000. The testbench compares all five outputs with an independent reference model for 40 clock cycles and then checks that *en* = 0 freezes the count.

The testbench also checks wrap-around: *up* and *down* over 40 cycles cross the 15/0 boundary more than twice.

5. Part C - Parameterized RTL design (Exercise 3)

Hard-coded widths make a module single-use. Verilog *parameter* and *generate* let one description serve many configurations.

Module	Parameters	Notes
register_n	N (default 8), ASYNC_RESET	N-bit register with enable; generate block selects reset style
counter_n	N (default 4), MOD (default 2**N), ASYNC_RESET	up/down counter, counts 0..MOD-1, synchronous clear, terminal count tc
shift_register_n	N (default 4), ASYNC_RESET	left shift register with serial_out

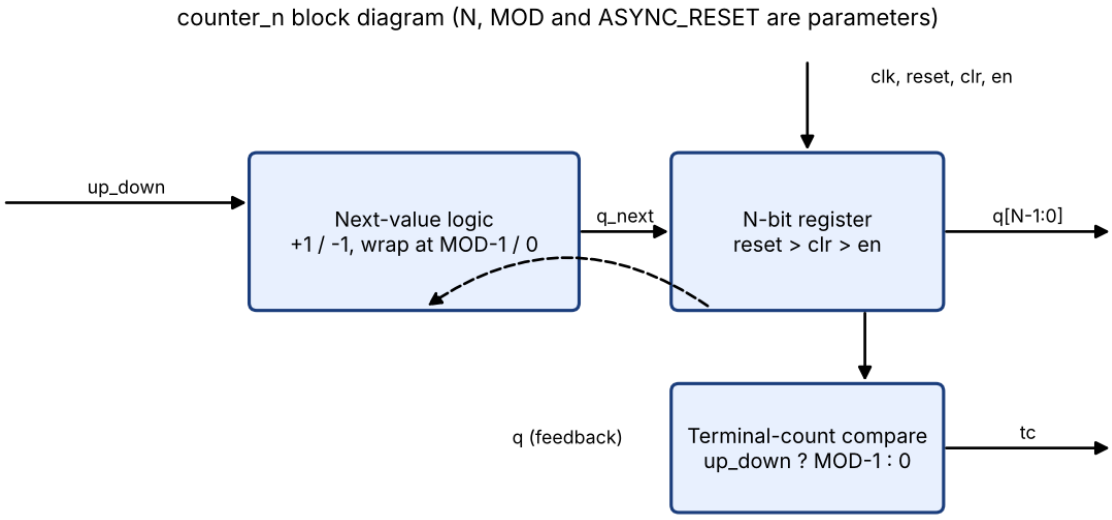


Figure 6: counter_n block diagram.

tb_param instantiates the modules at several configurations and checks them against reference models:

- registers with N = 4, 8 and 16 driven with random data, including a hold test with en = 0;
- counters: N = 3 (free-running, 0 to 7), N = 8 with MOD = 100 (0 to 99) and N = 4 with MOD = 10 (decade) counting up, down and up again;
- shift registers with N = 4, 8 and 16, all fed the 16-bit pattern A53C MSB first - after 16 clocks each holds the last N bits.

5.1 Simulation results

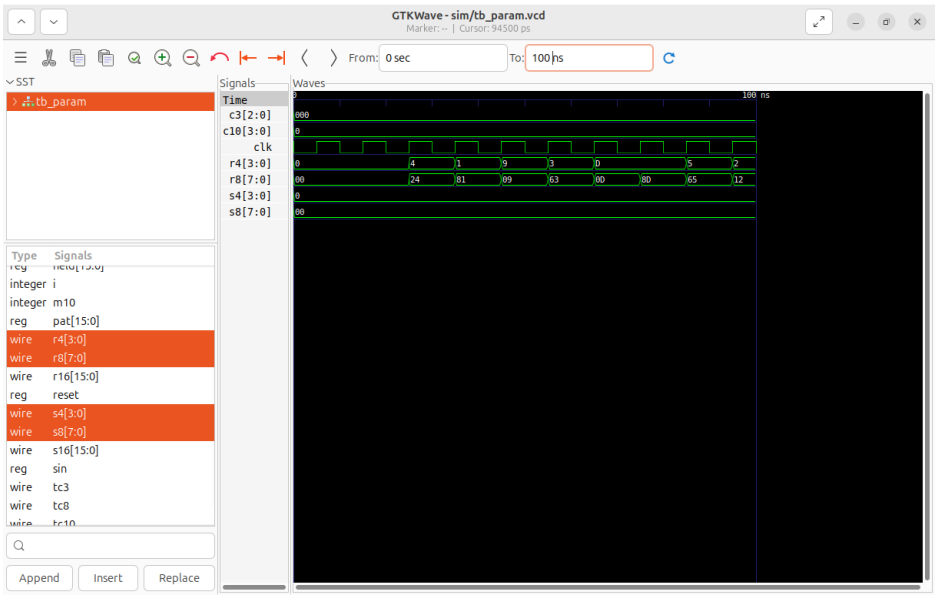


Figure 7: tb_param - registers: r4 and r8 loading new values (0 to 100 ns).

r4 always equals the low nibble of r8 (4/24, 1/81, 9/09, 3/63, D/0D, D/8D, 5/65, 2/12), which shows that the same module works correctly at both widths.

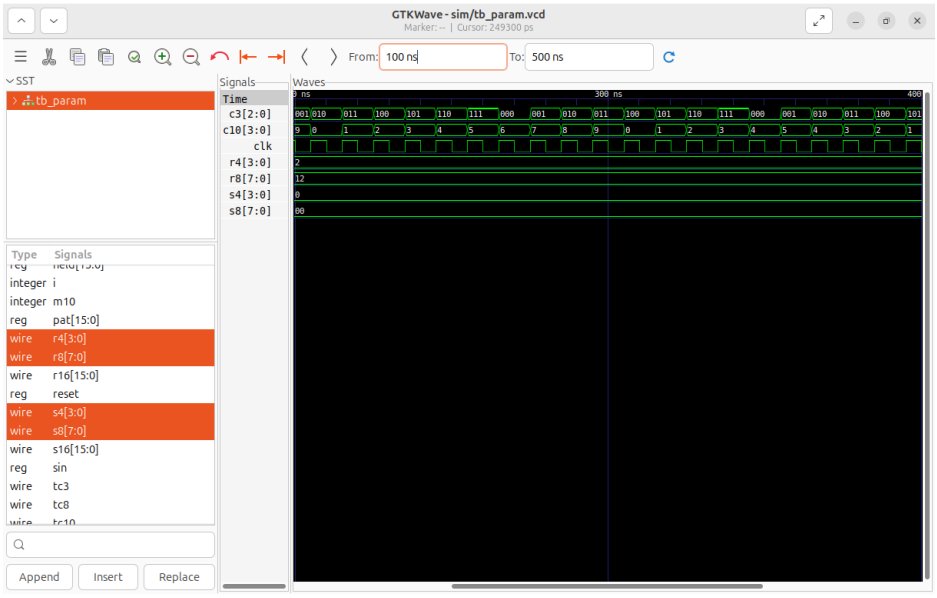


Figure 8: tb_param - counters c3 (N = 3) and c10 (N = 4, MOD = 10), 100 to 500 ns.

c3 counts 001 to 111 and wraps to 000. c10 counts 0 to 9 and wraps, then counts down (5, 4, 3, 2, 1) when *up_down* is low and goes back up afterwards. The 8-bit mod-100 counter c8 is checked by the testbench and is covered by the overall PASS result.

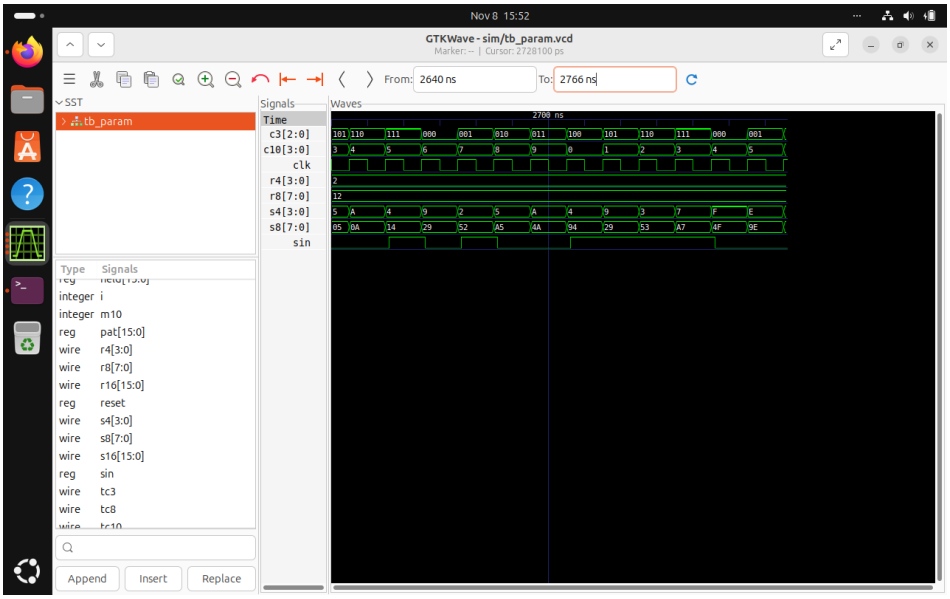


Figure 9: tb_param - shift registers s4 and s8 following the input sin (2640 to 2766 ns).

s8 moves 52, A5, 4A, 94, 29, 53, A7, 4F, 9E while *sin* feeds the last bits of the pattern A53C (0, 0, 1, 1, 1, 1, 0, 0); the next shift gives s8 = 3C and s4 = C, the low byte and low nibble of the pattern. That final step falls on the last clock edge and is confirmed by the testbench PASS.

6. Part D - Finite state machines (Exercise 4 and mini project)

The traffic light controller manages two roads, A and B, with the four states shown below. Each state lasts a configurable number of clock cycles (**GREEN_T** = 4 and **YELLOW_T** = 2 in simulation, so one full cycle is 12 clocks = 120 ns). An 8-bit timer counts clock cycles in the current state; when it reaches the state's duration the FSM advances. Lights are encoded as {red, yellow, green}: **100** = red, **010** = yellow, **001** = green.

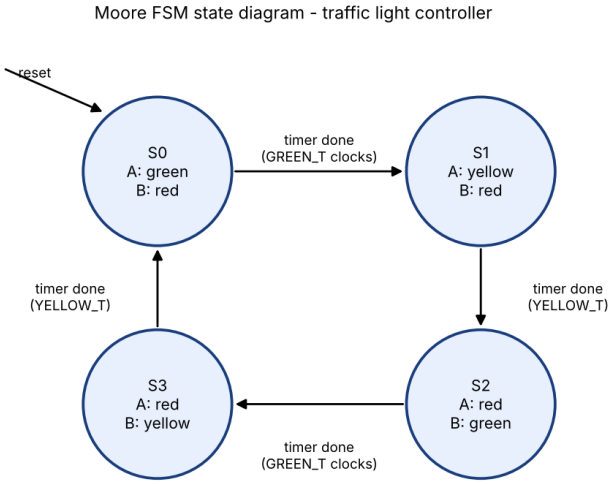


Figure 10: Moore FSM state transition diagram.

State	Encoding	Light A	Light B	Duration	Next state
S0	00	Green (001)	Red (100)	GREEN_T	S1
S1	01	Yellow (010)	Red (100)	YELLOW_T	S2
S2	10	Red (100)	Green (001)	GREEN_T	S3
S3	11	Red (100)	Yellow (010)	YELLOW_T	S0

State encoding is plain binary; the least significant bit is 1 in the two yellow states, which the RTL uses to pick YELLOW_T or GREEN_T. The design uses the standard three-part FSM structure: a state/timer register, combinational next-state logic and combinational output logic.

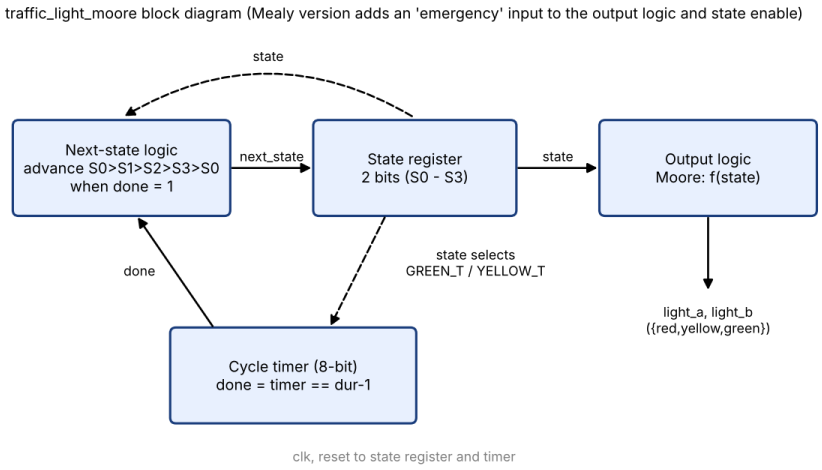


Figure 11: Block diagram of the traffic light controller.

6.1 Moore versus Mealy

	Moore (traffic_light_moore)	Mealy (traffic_light_mealy)
Outputs depend on	current state only	current state and the emergency input
Reaction to an input	changes only after a clock edge (needs an extra state)	same cycle, without waiting for a clock edge
Extra input	none	emergency: both lights red, FSM state and timer freeze
Glitch risk	low - outputs come from registered state	input glitches can reach the outputs

6.2 Simulation results

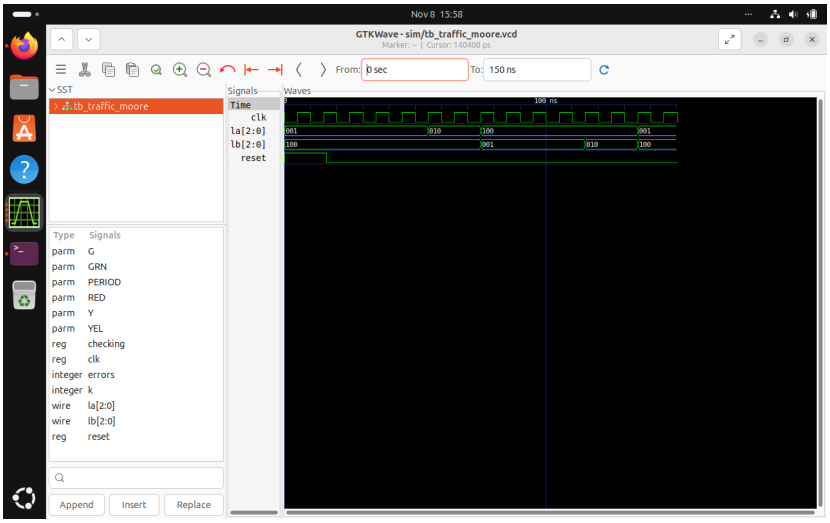


Figure 12: tb_traffic_moore - normal operation (0 to 150 ns).

After reset is released (about 16 ns) light A is green (001) for four clocks, yellow (010) for two, then red (100) for six clocks. Light B is red (100) while A is green and yellow, green for four clocks and yellow for two. The two roads are never green at the same time, and the 12-clock cycle repeats. The testbench compares both lights with a reference model every cycle for three full cycles.

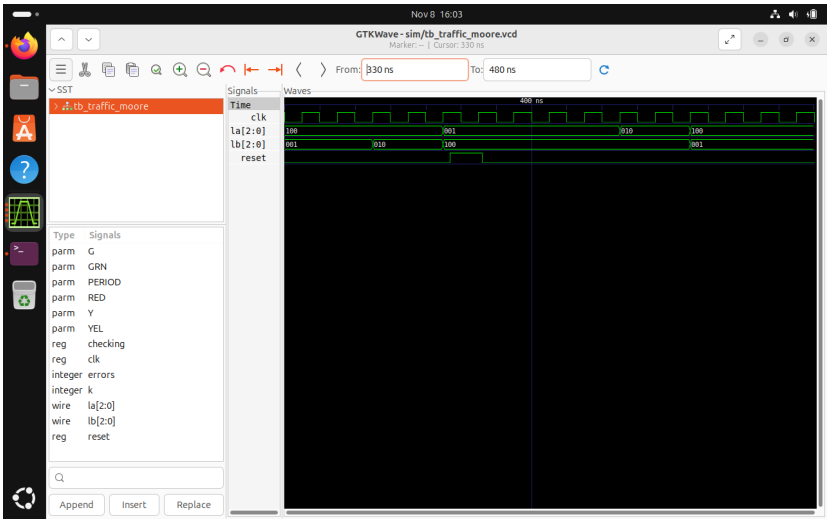


Figure 13: tb_traffic_moore - reset applied during operation (330 to 480 ns).

Reset is pulsed at about 376 ns, just as the controller enters S0 at the end of the third cycle. The state register and timer are held at S0/0, so light A stays green for one extra clock (50 ns instead of 40 ns) before the next sequence begins, and the outputs show the S0 values (A green, B red) while reset is high. The testbench also checks the lights immediately after reset rises.

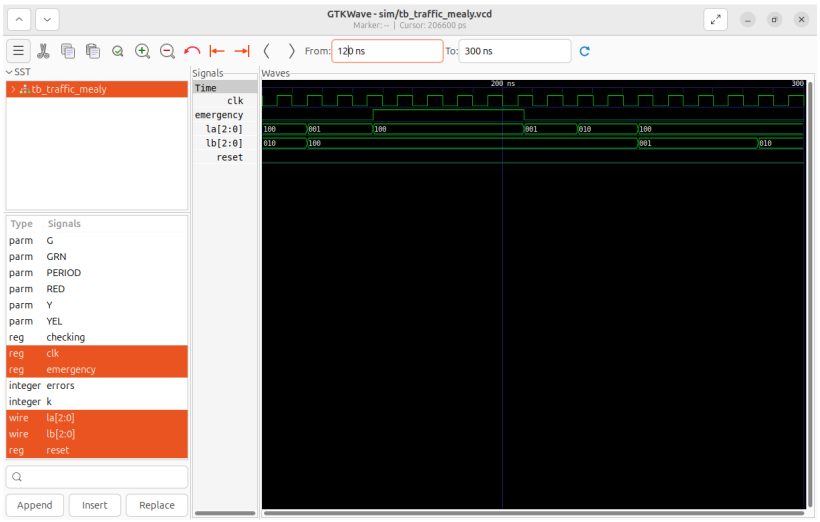


Figure 14: tb_traffic_mealy - emergency input (120 to 300 ns).

At about 158 ns *emergency* goes high in the middle of a clock cycle and light A switches from green (001) to red (100) at once; light B is already red, so both lights are red. The state and timer stay frozen while *emergency* is high. At about 207 ns it goes low and light A returns to green, then continues with yellow and red as the sequence resumes where it stopped. This immediate, clock-independent reaction is the Mealy behaviour.

7. Part E - Timing analysis (Exercise 5)

Synchronous circuits work only if the data input is stable around the active clock edge.

- **Setup time (t_su)**: the minimum time D must be stable *before* the active clock edge.
- **Hold time (t_h)**: the minimum time D must remain stable *after* the active clock edge.
- **Clock period**: T must satisfy T >= t_clk-to-q + t_logic + t_su (plus clock skew), otherwise the next flip-flop captures the wrong value.
- **Clock skew**: the difference in clock arrival time between two flip-flops; it reduces the setup margin or the hold margin depending on its sign.
- A violation can leave the flip-flop in a metastable state; in simulation this appears as X.

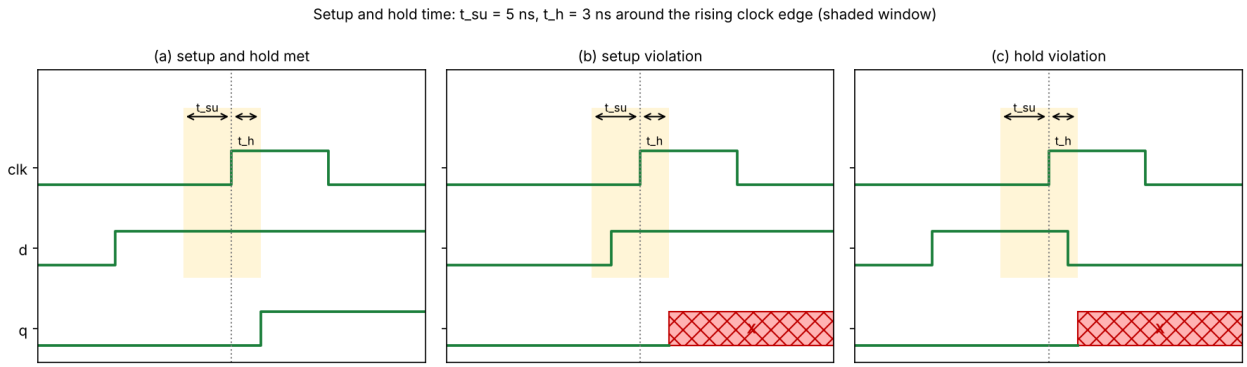


Figure 15: Setup and hold timing diagram (t_su = 5 ns, t_h = 3 ns).

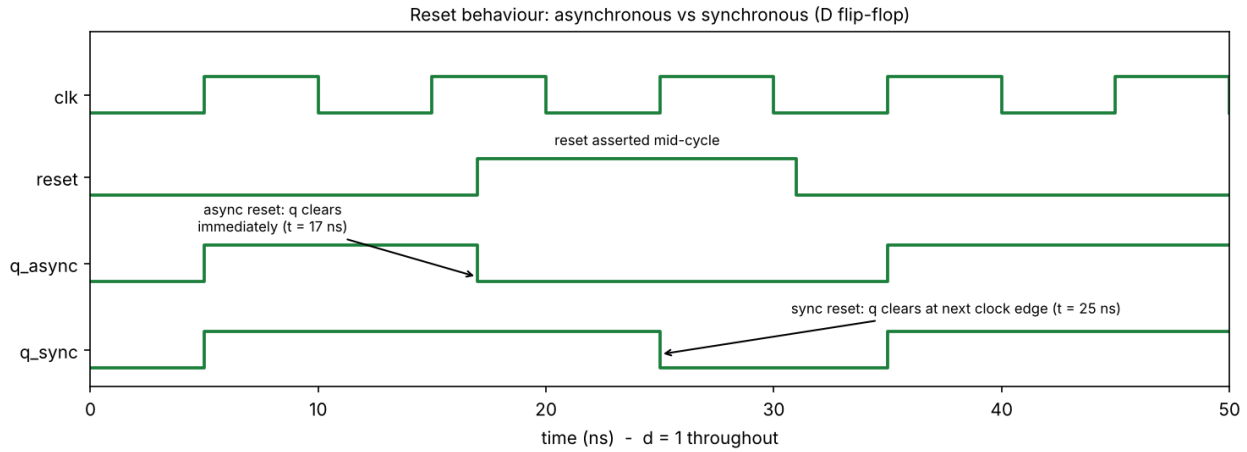


Figure 16: Asynchronous versus synchronous reset timing.

7.1 Simulated setup and hold violations

tb_timing_demo uses a behavioral flip-flop model with t_su = 2 ns and t_h = 1 ns and a 10 ns clock (rising edges at 5, 15, 25, 35 and 45 ns). The model reports violations and drives *q* to X.

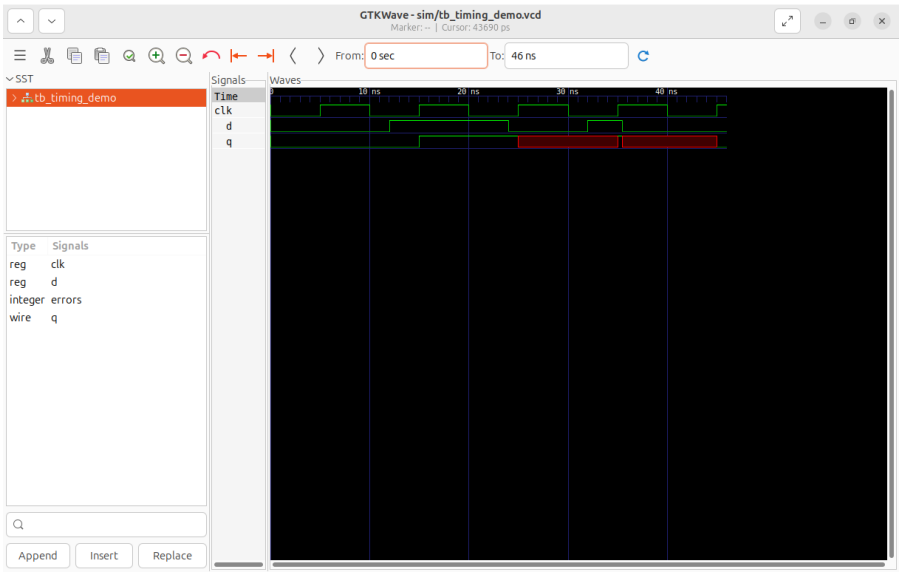


Figure 17: tb_timing_demo - setup and hold violations.

Time	Event	Margin	Result
12 ns	d rises 3 ns before the edge at 15 ns	setup margin 3 ns >= 2 ns	q = 1 at 15 ns (correct capture)
24 ns	d falls 1 ns before the edge at 25 ns	1 ns < t_su = 2 ns	setup violation, q = X
35.5 ns	d falls 0.5 ns after the edge at 35 ns	0.5 ns < t_h = 1 ns	hold violation, q = X

The testbench passes only if exactly one setup violation and one hold violation are detected and q recovers to the correct value on the next clean edge (45 ns).

8. Part F - Testbench development and verification summary (Exercise 6)

Every testbench follows the same reusable structure:

- **Clock generation**: *always #5 clk = ~clk;* (10 ns period, 100 MHz).
- **Reset generation**: reset is asserted at the start, and mid-cycle in the reset tests, then released away from the clock edge to avoid races.
- **Multiple stimulus patterns**: directed values, random data ($random), up/down switching, enable on/off, emergency pulses and button presses.
- **Automatic checking**: outputs are compared with expected values (reference models) one cycle after the clock edge, with !== so X values are caught; errors are counted.
- **Waveform output**: $dumpfile / $dumpvars write a VCD file into sim/ for GTKWave.
- **Result line**: RESULT: PASS or RESULT: FAIL so run_all.sh can summarise automatically.

Testbench	DUT	What is checked	Result
tb_d_flip_flop	d_flip_flop (async and sync)	load, async clear, sync clear on next edge	PASS
tb_register	register8 (async and sync)	load, hold with en = 0, mid-cycle reset	PASS
tb_shift_register	shift_register_n, N = 4	SIPO value 1011, SISO bit order, flush	PASS
tb_counters	5 counters	40-cycle sequences vs reference model, enable freeze	PASS
tb_param	register_n, counter_n, shift_register_n	N = 3/4/8/16, MOD = 10 and 100, up/down	PASS
tb_traffic_moore	traffic_light_moore	36-cycle sequence, async reset	PASS
tb_traffic_mealy	traffic_light_mealy	sequence, immediate emergency response, freeze and resume	PASS
tb_timing_demo	behavioral timing model	1 setup + 1 hold violation, recovery	PASS
tb_stopwatch	stopwatch_controller	run, pause, resume, clear, minute rollover	PASS

Summary: 9 testbenches compiled and ran with no errors; all 9 printed RESULT: PASS.

9. Exercise 7 - Digital stopwatch controller

The stopwatch combines an FSM, counters, registers and parameterized modules. **start_stop** is edge-detected so one press toggles between RUN and PAUSE; **clear** zeroes the time while the stopwatch is not running.

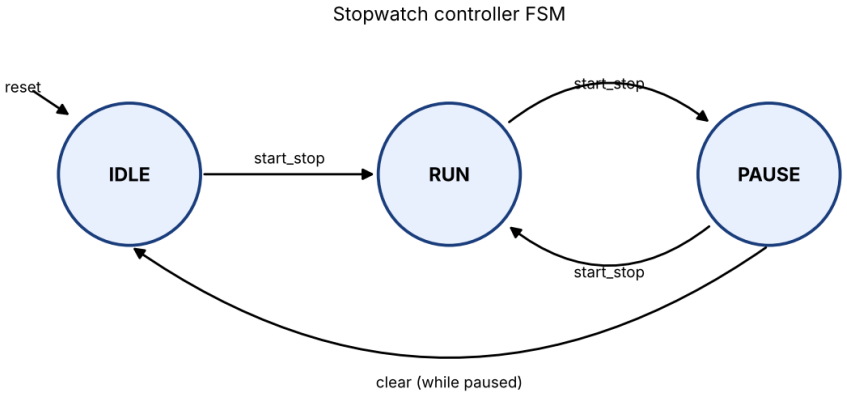


Figure 18: Stopwatch controller FSM.

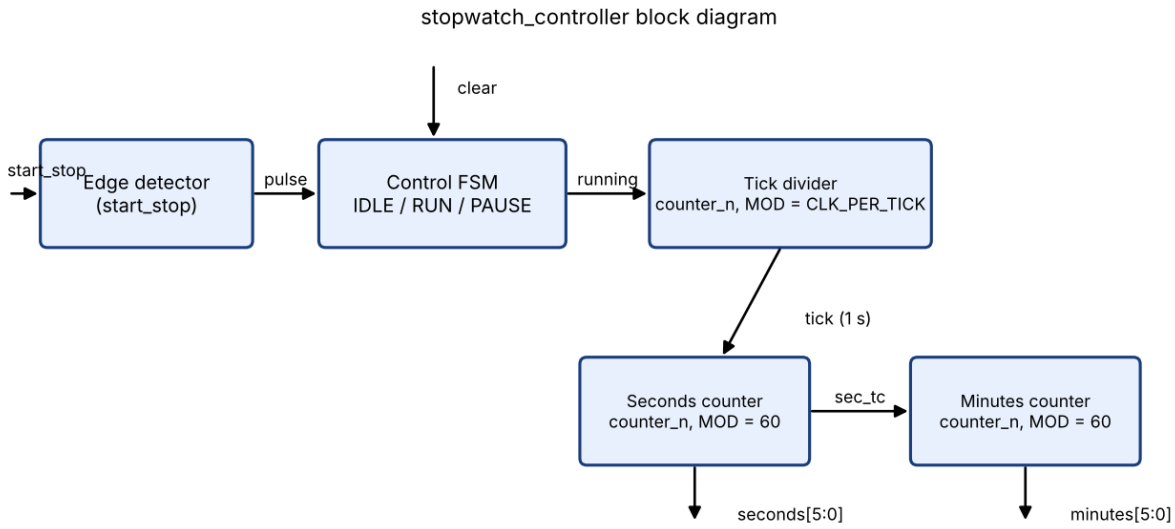


Figure 19: Stopwatch controller block diagram.

Parameter	Default	Meaning
CLK_PER_TICK	50,000,000	clock cycles per second (50 MHz clock); the testbench uses 4 so a 'second' is 4 clocks
TICK_W	26	width of the tick divider counter

The tick divider is a counter_n with MOD = CLK_PER_TICK that runs only while *running* is high. Its terminal count enables the seconds counter (counter_n, N = 6, MOD = 60), whose terminal count in turn enables the minutes counter (also MOD = 60). One clear signal resets all three counters.

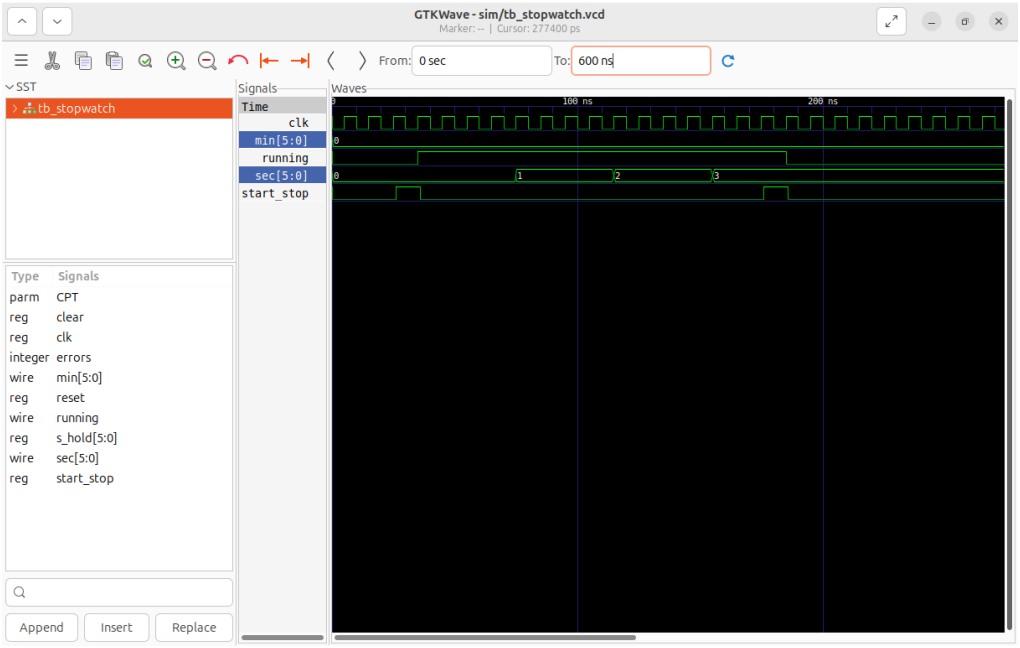


Figure 20: tb_stopwatch - start, count and pause (0 to 600 ns).

A start_stop pulse at about 35 ns moves the FSM to RUN (*running* goes high). *sec* then increments every 40 ns (4 clock cycles = one simulated second): 1 at about 75 ns, 2 at 115 ns and 3 at 155 ns. A second pulse at about 185 ns pauses the watch: *running* goes low and *sec* holds at 3. The rest of the testbench (resume, clear, and a 60-second run that increments *min* to 1 with *sec* back at 0) occurs after the time range shown and is checked automatically; the testbench reports PASS.

10. RTL coding guidelines followed

- Non-blocking assignments (<=) in all sequential always blocks; blocking assignments (=) only in combinational always @(*) blocks.
- Every sequential block has a clock and a reset; reset values are explicit.
- Combinational and sequential logic are kept in separate always blocks (state register, next-state logic, output logic).
- Widths are parameters (N, MOD) instead of hard-coded numbers; generate blocks select sync or async reset.
- All case statements have a default branch, so next-state and output logic contain no latches.
- States and light codes are named with localparam instead of magic numbers.
- One module per file, file name equals module name, consistent indentation and comments.
- Simulation-only code (delays, $display, behavioral timing model) is kept in tb/ and never in rtl/.
- Project is organised into rtl/, tb/, sim/ and docs/ with a README and run script.

11. Conclusion and deliverables checklist

Task 2 delivers a library of tested sequential building blocks - flip-flops, registers, shift registers, five counters, parameterized register/counter/shift-register modules, Moore and Mealy traffic light controllers and a configurable stopwatch - together with self-checking testbenches and waveform evidence. These blocks will be reused in the datapath and control unit of the 32-bit RISC processor capstone project, and the testbench structure leads into the RTL verification work of Task 3.

Deliverable	Location
Verilog RTL source files	rtl/
Testbench files	tb/
Simulation output (VCD, logs)	sim/
GTKWave screenshots	docs/screenshots/ (12 images, reproduced in this document)
FSM state transition diagrams	Figures in sections 6 and 9
RTL block diagrams	Figures in sections 3, 5, 6 and 9
Timing diagrams	Section 7
Counter design documentation	Section 4
GitHub repository and README.md	github.com/JYOTHIRMAYD (vlsi-task2-sequential-rtl)
Project documentation (PDF)	this document
Daily progress log	docs/daily_progress_log.md

Appendix - RTL source listings

A.1 d_flip_flop.v

```
`timescale 1ns/1ps
// D flip-flop with selectable reset style.
// ASYNC_RESET = 1 : asynchronous active-high reset (matches the starter code)
// ASYNC_RESET = 0 : synchronous active-high reset
module d_flip_flop #(
    parameter ASYNC_RESET = 1
)(
    input  wire clk,
    input  wire reset,
    input  wire d,
    output reg  q
);
generate
    if (ASYNC_RESET) begin : g_async
        always @(posedge clk or posedge reset) begin
            if (reset) q <= 1'b0;
            else       q <= d;
        end
    end else begin : g_sync
        always @(posedge clk) begin
            if (reset) q <= 1'b0;
            else       q <= d;
        end
    end
endgenerate
endmodule
```

A.2 counter_n.v

```
`timescale 1ns/1ps
// Parameterized up/down counter with modulus.
//   N    : width
//   MOD  : counts 0..MOD-1 (default 2**N = free-running binary counter)
//   up_down: 1 = count up, 0 = count down
//   clr  : synchronous clear
//   tc   : terminal count (MOD-1 when counting up, 0 when counting down)
module counter_n #(
    parameter N = 4,
    parameter MOD = (1 << N),
    parameter ASYNC_RESET = 1
)(
    input  wire         clk,
    input  wire         reset,
    input  wire         clr,
    input  wire         en,
    input  wire         up_down,
    output reg  [N-1:0] q,
    output wire         tc
);
    wire [N-1:0] last = MOD - 1;
    assign tc = up_down ? (q == last) : (q == {N{1'b0}});

    wire [N-1:0] q_next = up_down ? ((q == last) ? {N{1'b0}} : q + 1'b1)
                                  : ((q == {N{1'b0}}) ? last : q - 1'b1);
generate
    if (ASYNC_RESET) begin : g_async
        always @(posedge clk or posedge reset) begin
            if (reset)    q <= {N{1'b0}};
            else if (clr) q <= {N{1'b0}};
            else if (en)  q <= q_next;
        end
    end else begin : g_sync
        always @(posedge clk) begin
            if (reset)    q <= {N{1'b0}};
            else if (clr) q <= {N{1'b0}};
            else if (en)  q <= q_next;
        end
    end
endgenerate
endmodule
```

A.3 traffic_light_moore.v

```
`timescale 1ns/1ps
// Moore FSM traffic light controller (two roads, A and B).
// Outputs depend ONLY on the current state.
//   S0: A green,  B red     (GREEN_T  clock cycles)
//   S1: A yellow, B red     (YELLOW_T clock cycles)
//   S2: A red,    B green   (GREEN_T  clock cycles)
//   S3: A red,    B yellow  (YELLOW_T clock cycles)
// Light encoding: {red, yellow, green}
module traffic_light_moore #(
    parameter GREEN_T  = 6,
    parameter YELLOW_T = 2
)(
    input  wire       clk,
    input  wire       reset,     // async, active high
    output reg  [2:0] light_a,
    output reg  [2:0] light_b
);
    localparam [1:0] S0 = 2'd0, S1 = 2'd1, S2 = 2'd2, S3 = 2'd3;
    localparam [2:0] RED = 3'b100, YEL = 3'b010, GRN = 3'b001;

    reg  [1:0] state, next_state;
    reg  [7:0] timer;

    wire [7:0] dur  = state[0] ? YELLOW_T : GREEN_T;   // odd states = yellow
    wire       done = (timer == dur - 1);

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            state <= S0;
            timer <= 8'd0;
        end else begin
            state <= next_state;
            timer <= done ? 8'd0 : timer + 8'd1;
        end
    end

    always @(*) begin
        next_state = state;
        if (done) begin
            case (state)
                S0:      next_state = S1;
                S1:      next_state = S2;
                S2:      next_state = S3;
                S3:      next_state = S0;
                default: next_state = S0;
            endcase
        end
    end

    // output logic (Moore: function of state only)
    always @(*) begin
        case (state)
            S0:      begin light_a = GRN; light_b = RED; end
            S1:      begin light_a = YEL; light_b = RED; end
            S2:      begin light_a = RED; light_b = GRN; end
            S3:      begin light_a = RED; light_b = YEL; end
            default: begin light_a = RED; light_b = RED; end
        endcase
    end
endmodule
```

A.4 stopwatch_controller.v

```
`timescale 1ns/1ps
// Configurable digital stopwatch (Exercise 7).
//   CLK_PER_TICK : clock cycles per 1 second (e.g. 50_000_000 for 50 MHz)
//   TICK_W       : width of the tick divider (must hold CLK_PER_TICK-1)
// start_stop : press to toggle RUN/PAUSE (edge-detected)
// clear      : zeroes the time while NOT running
// FSM: IDLE --start--> RUN <--start/stop--> PAUSE --clear--> IDLE
module stopwatch_controller #(
    parameter CLK_PER_TICK = 50_000_000,
    parameter TICK_W       = 26
)(
    input  wire       clk,
    input  wire       reset,
    input  wire       start_stop,
    input  wire       clear,
    output wire [5:0] seconds,
    output wire [5:0] minutes,
    output wire       running
);
    localparam [1:0] IDLE = 2'd0, RUN = 2'd1, PAUSE = 2'd2;

    reg [1:0] state, next_state;
    reg       ss_d;
    wire      ss_pulse = start_stop & ~ss_d;      // rising-edge detect

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            state <= IDLE;
            ss_d  <= 1'b0;
        end else begin
            state <= next_state;
            ss_d  <= start_stop;
        end
    end

    always @(*) begin
        next_state = state;
        case (state)
            IDLE:    if (ss_pulse) next_state = RUN;
            RUN:     if (ss_pulse) next_state = PAUSE;
            PAUSE:   if (clear)         next_state = IDLE;
                     else if (ss_pulse) next_state = RUN;
            default: next_state = IDLE;
        endcase
    end

    assign running = (state == RUN);
    wire clear_cmd = clear & ~running;

    wire tick_tc, sec_tc;

    counter_n #(.N(TICK_W), .MOD(CLK_PER_TICK)) u_tick (
        .clk(clk), .reset(reset), .clr(clear_cmd), .en(running),
        .up_down(1'b1), .q(), .tc(tick_tc)
    );
    wire tick = tick_tc & running;

    counter_n #(.N(6), .MOD(60)) u_sec (
        .clk(clk), .reset(reset), .clr(clear_cmd), .en(tick),
        .up_down(1'b1), .q(seconds), .tc(sec_tc)
    );

    counter_n #(.N(6), .MOD(60)) u_min (
        .clk(clk), .reset(reset), .clr(clear_cmd), .en(tick & sec_tc),
        .up_down(1'b1), .q(minutes), .tc()
    );
endmodule
```